Lecture 7: Programming

James Sears*
AFRE 891/991 FS 26
Michigan State University

*Parts of these slides are adapted from **"Data Science for Economists"** by Grant McDermott and **"Advanced Data Analytics"** by Nick Hagerty.

Table of Contents

1. Prologue
2. If/Else Statements
3. For Loops
4. Functions
5. Indirection and Name Injection
6. Vectorization
7. Parallelization

Prologue

Programming

So far in class we've learned how to do a lot of things in R, but we can exponentially increase our data analytics skills (and how quickly we get things done) by learning some **programming**.

- Write **custom functions** to execute specific tasks
  - Scrape all Yellowpages business links for a given search term in hundreds of different cities
- **Conditionally** define variables or execute different tasks
  - Create a variable conditional on another variables' values
- Perform a repeated task by **looping over values**
  - Create a set of state-level dummy variables from state FIPS codes
- Run tasks efficiently **in parallel**
  - Calculate parcel or farm-level measures of precipitation and temperature

Programming

Packages we'll use today:

```
pacman::p_load(dslabs, tidyverse, furrr, tictoc, future, progressr)
```

And let's load in the `murders` data from the `dslabs` package:

```
data(murders)
```

If/Else Statements

If/Else Statements

If/else statements are a type of **conditional expression**.

- Check to see if a logical condition is True
- If True, do a thing
- If False:
    - Do a different thing,
    - Do nothing, or
    - Check *another* condition, do a thing if True, etc.

If/Else Statements

For example: print the reciprocal of `a`, unless `a` is 0.

```
a = 0

if(a ≠ 0) {

  print(1 / a)

} else {

  print("Reciprocal does not exist.")

}
```

```
## [1] "Reciprocal does not exist."
```

Statements like this are used for **control flow** of your code.

- Used all the time in software development
- Used occasionally in data analysis, more often in custom functions and packages.

If/Else Statements

You can also link together multiple condition with `else if`s.

```
if(a > 0) {

  print("a is Positive")

} else if (a < 0){

  print("a is Negative")

} else {

  print("a is Zero")
}
```

```
## [1] "a is Zero"
```

If/Else Statements

A related function that you *will* use all the time in data analysis: `ifelse`.

syntax: `ifelse(CONDITION, ACTION_IF_TRUE, ACTION_IF_FALSE)`

- `CONDITION`: a logical condition `ACTION_IF_TRUE`: *what to do if the condition is true* `ACTION_IF_FALSE`: what to do if the condition is false

For example:

```
a = 0
ifelse(a > 0, 1/a, NA)
```

```
## [1] NA
```

If/Else Statements

syntax: `ifelse(CONDITION, ACTION_IF_TRUE, ACTION_IF_FALSE)`

`ifelse` is particularly useful because it is **vectorized** and can be applied over **a vector of elements all at once**

For example, to change negative numbers to missing:

```
b = c(0, 1, 2, -3, 4)
ifelse(b < 0, NA, b)
```

```
## [1]  0  1  2 NA  4
```

If/Else Statements

syntax: `ifelse(CONDITION, ACTION_IF_TRUE, ACTION_IF_FALSE)`

`ifelse` is particularly useful because it is **vectorized** and can be applied over **a vector of elements all at once**

Or for adding a conditional variable - for example, whether or not a state is Michigan

```
murders ← murders %>% mutate(
    is_michigan = ifelse(state == "Michigan", "Is Michigan", "Is Not
Michigan")
                    )
murders[c(1, 23:26),]
```

```
##          state abb        region population total     is_michigan
## 1      Alabama  AL         South    4779736   135 Is Not Michigan
## 23    Michigan  MI North Central    9883640   413     Is Michigan
## 24   Minnesota  MN North Central    5303925    53 Is Not Michigan
## 25 Mississippi  MS         South    2967297   120 Is Not Michigan
```

if_else

Note that tidyverse has its *own, expanded* version of this function: `if_else()`. Benefits include

- Can use the optional argument `missing` to handle NA values of the condition

```
x ← c(-2:0, NA, 2)
ifelse(x < 0, "Negative", "Positive")
```

```
## [1] "Negative" "Negative" "Positive" NA         "Positive"
```

```
if_else(x < 0, "Negative", "Positive", "Missing")
```

```
## [1] "Negative" "Negative" "Positive" "Missing"  "Positive"
```

if_else

Note that tidyverse has its *own, expanded* version of this function: `if_else()`. Benefits include

- Preserving data types!

```
x ← factor(sample(letters[1:5], 10, replace = TRUE))
ifelse(x %in% c("a", "b", "c"), x, NA)
```

```
##  [1]  3  1  2 NA  1  3 NA  2  1  3
```

```
if_else(x %in% c("a", "b", "c"), x, NA)
```

```
##  [1] c    a    b    <NA> a    c    <NA> b    a    c
## Levels: a b c e
```

case_when()

While it's technically possible to use nested ifelses, friends don't let friends nest ifelses.

Instead, use **dplyr's** `case_when()`

- Checks first condition, any elements with TRUE get assigned value + exit
  - Check second condition for remaining elements, TRUE cases get value + exit
  - Repeat until end of the tree (`TRUE`) and assign them to a value

```
x ← 1:10
## dplyr::case_when()
case_when(
  x ⩽ 3 ~ "small",
  x ⩽ 7 ~ "medium",
  TRUE ~ "big"      # Default
value
  )
```

```
##  [1] "small"  "small"  "small"
"medium" "medium" "medium"
"medium" "big"
##  [9] "big"    "big"
```

case_when()

Works great within `mutate()` as well!

```
murders ← murders %>% mutate(
    my_opinion = case_when(
      state == "Michigan" ~ "Great State",
      state %in% c("California", "Hawaii") ~ "Also Solid State",
      state == "Missouri" ~ "More like Misery am I right",
      TRUE ~ "A State")
    )
murders[c(1, 5, 12, 23, 26, 38),c(1,7)]
```

```
##         state                    my_opinion
## 1     Alabama                       A State
## 5  California             Also Solid State
## 12     Hawaii             Also Solid State
## 23   Michigan                  Great State
## 26   Missouri More like Misery am I right
## 38     Oregon                       A State
```

While

While Loops

While we're on the topic... let's consider a **while loop**:

- Continue to do an action *while* a condition is met
- Make sure to increment i else the loop will run forever!

```
i ← 1
while  (i < 8){
  print(paste(i, "is less than 8"))
  i ← i + 1
}
```

```
## [1] "1 is less than 8"
## [1] "2 is less than 8"
## [1] "3 is less than 8"
## [1] "4 is less than 8"
## [1] "5 is less than 8"
## [1] "6 is less than 8"
## [1] "7 is less than 8"
```

While and If

We can nest `if` and `break` into a `while` to **stop the loop even if the while condition is TRUE**

```
num ← 1

while (num < 8){
  print(paste(num/2, "is half of", num))
  num ← num + 1

  if (floor(num/2) == 2) {
    break
  }
}
```

```
## [1] "0.5 is half of 1"
## [1] "1 is half of 2"
## [1] "1.5 is half of 3"
```

For Loops

Abstraction

Often you will have tasks where you find yourself copying and pasting your code to do the same thing n times, with only minor tweaks each time.

Q: What's wrong with that?

- Annoying (especially if n is large)
- Hard to change later if needed
- Prone to errors/bugs

Instead, you can **abstract** your code: define it once, and run it multiple times. The rest of this lecture covers tools for abstraction in different situations.

A good rule to aim for is to **never copy-and-paste more than twice.** If you're pasting more than that, abstract it instead!

Abstraction Methods

There are several different methods for code abstraction that we'll go over:

1. **For loops:** when you want to repeat the same code for **different values of a variable or vector**
2. **Functions:** when you want to repeat the same code for potentially **different values of all arguments/variables** or with **different settings/samples**
3. **Vectorization and Functionals:** when you want to **repeat a function** over **different values of arguments**

For Loops

The **for loop** is a simple tool for **iteration**

```
for (INDEX in RANGE){
  action(INDEX)
}
```

- `INDEX` the name of the index you want to use (often `i` but can be anything)
  - Often `i` but can be literally anything R-legal
- `RANGE` the vector of values to iterate over
  - Can be numbers, characters, or objects
- `action` the stuff to do for each index value in the specified range

For Loops

Good use for a for loop: repeat the same thing with minor tweaks.

```
for (i in 1:6){
  print(paste0("It is ", i, " O'Clock."))
}
```

```
## [1] "It is 1 O'Clock."
## [1] "It is 2 O'Clock."
## [1] "It is 3 O'Clock."
## [1] "It is 4 O'Clock."
## [1] "It is 5 O'Clock."
## [1] "It is 6 O'Clock."
```

For Loops

You can also combine for loops with if-else:

```
states ← c("Indiana", "Michigan", "Colorado", "michigan")
for (j in states){
  if (j == "Michigan|michigan"){
      print("This is Michigan")

   } else {
     print("This is not Michigan")
  }
}
```

```
## [1] "This is not Michigan"
## [1] "This is not Michigan"
## [1] "This is not Michigan"
## [1] "This is not Michigan"
```

For Loops

Suppose you wanted to calculate the mean of the numeric variables in `murders` and the murder rate. We could manually type and copy-paste:

```
murders ← mutate(murders, rate = total/population * 1e5)
mean(murders$total)
```

```
## [1] 184.3725
```

```
mean(murders$population)
```

```
## [1] 6075769
```

```
mean(murders$rate)
```

```
## [1] 2.779125
```

For Loops

Or we could avoid copy-past errors and use a for loop:

```
for (var in c("total", "population", "rate")){
print(mean(murders[[var]]))
}
```

```
## [1] 184.3725
## [1] 6075769
## [1] 2.779125
```

For Loops

We an also loop over an **object in memory:**

```
numeric_col ← c("total", "population", "rate")
for (var in numeric_col){
print(mean(murders[[var]]))
}
```

```
## [1] 184.3725
## [1] 6075769
## [1] 2.779125
```

For Loops

Or **assign output to memory** too

```
numeric_col ← c("total", "population", "rate")
means ← vector() # initiate an empty vector
for (var in numeric_col){
means[[var]] ← mean(murders[[var]])
}
```

There is one technical problem with this code. The vector storing the output **"grows" at each iteration**, which can make the loop **very slow**.

For Loops

Better: give your empty vector the **right length** *before* starting.

```
means ← vector("numeric", length = length(numeric_col)) # initiate an
empty vector with same length
for (i in 1:length(numeric_col)){
  col_num ← which(colnames(murders) == numeric_col[i])
  means[[i]] ← mean(murders[[col_num]])
}
```

For Loops: Caveat

For-loops are actively **discouraged in R programming**.

- We're covering them because the concepts are foundational.
- But R has nicer ways to iterate, called **vectorization**.
    - Often *exponentially* faster
- To do proper vectorization, we first need to know how to **write functions**.

Functions

Functions

We've already seen a **multitude of functions** in R

- pre-packaged with base R
- loaded by different packages (e.g. `dplyr::mutate()`)

Regardless of where they come from, the all follow the same basic syntax:

```
function_name(ARGUMENTS)
```

Custom Functions

While we will often use pre-made functions, you can - and should! - write your own functions too. This is easy to do with the generic `function()` function.[1]

If you only have a short function, you can write it all on a **single line:**

```
function(ARGUMENTS) OPERATIONS
```

[1] Yes, it's a function that let's you write functions. Very meta.

Custom Functions

Oftentimes we want our function code to span **multiple lines**. In this case we can use brackets around the function's operations:

```
function(ARGUMENTS) {
  OPERATIONS
  return(VALUE)
}
```

Custom Functions

Rather than write **anonymous** functions, **name your functions** to assign them to memory and reuse them throughout your file:

```
my_func ← function(ARGUMENTS) {
    OPERATIONS
    return(VALUE)
  }
```

Try to give your functions short, pithy names that are

- Informative to you
- Clear to anyone else who might read the code

Building Custom Functions

Let's start with a basic function: calculate a **number's square**.[2]

```
square ←       # function name
  function(x){ # the arguments of our function (here just one)
  x^2          # the operation(s) that our function performs
  }
```

Testing:

[2] I want to note that this **isn't a useful function**. R's arithmetic functions already handle vectorised exponentiation and do so very efficiently.

Specifying Return Values

If we **don't specify return values**, we get back the output from the **last computed step**

```
square_plus3 ← function(x){
  x_sq ← x^2 # assign squared value as intermediate object
  x_sq + 3
}
square_plus3(3)
```

```
## [1] 12
```

Specifying Return Values

Note that if instead you save the final step to memory, you instead **get nothing**

```
square_plus3 ← function(x){
  x_sq ← x^2 # assign squared value as intermediate object
  x_sq_pl_3 ← x_sq + 3
}
square_plus3(3)
```

Specifying Return Values

We can **specify return values** with `return()`

- Helpful when our function performs a bunch of **intermediate steps**
- Necessary when you save the last step to memory

```
square_plus3 ← function(x){
  x_sq ← x^2 # assign squared value as intermediate object
  x_sq_pl_3 ← x_sq + 3
  text ← paste("The Square of", x, "+3 is", x_sq_pl_3)
return(x_sq_pl_3)
}
square_plus3(2)
```

```
## [1] 7
```

Specifying Return Values

```
square_plus3(3)
```

```
## [1] 12
```

Note that the intermediate objects **don't stay in memory** - they're automatically removed as soon as the function is done running.

Specifying Return Values

If we want to return **multiple objects** from our function, we need to either

1. Return a List

```
square_ret_list ← function(x){
  x_sq ← x^2 # assign squared value as intermediate object
  res ← list(value = x, val_squared = x_sq)
  return(res)
  }

square_ret_list(3)
```

```
## $value
## [1] 3
##
## $val_squared
## [1] 9
```

Specifying Return Values

If we want to return **multiple objects** from our function, we need to either

2. Build a data frame (a tidy solution!)

```
square_ret_df ← function(x){
  x_sq ← x^2 # assign squared value as intermediate object
  res ← data.frame(value = x, val_squared = x_sq)
  return(res)
  }

square_ret_df(3)
```

```
##   value val_squared
## 1     3           9
```

Default Argument Values

We can also assign **default argument values**

- Allows for all/any arguments to be optional
- Uses the supplied value when supplied
- Uses provided default value when not

Suppose we wanted to expand our function to do any exponent and not just squares:

```
raise_power ← function(x = 2, power = 2){
  res ← data.frame(
      value = x,
      power = power,
      value_raised = x^power
  )
  return(res)
}

raise_power(5)
```

Default Argument Values

Setting default values doesn't affect typical function usage:

```
raise_power(x = 5, power = 3) # uses specified values
```

```
##   value power value_raised
## 1     5     3          125
```

But now any argument that we omit will **use the default values** and the function will run:

```
raise_power() # uses default values of x and power = 2
```

```
##   value power value_raised
## 1     2     2            4
```

Default Argument Values

Setting default values doesn't affect typical function usage:

```
raise_power(x = 5, power = 3) # uses specified values
```

```
##   value power value_raised
## 1     5     3           125
```

Without supplying argument values, our previous function wouldn't have worked:

```
square()
```

```
## Error in square():
## ! argument "x" is missing, with no default
```

Indirection and Name Injection

Indirection

A common use-case for custom functions is **iterating over variables**

- Repeat a cleaning task over multiple variables in a data frame
- Run analysis with a different dependent variable

For example, let's go back to our `square` function. By default it applies over an entire vector:

```
square(murders$rate)
```

```
##  [1]   7.9773697   7.1566199  13.1734682  10.1722092  11.3848093
1.6704350
##  [7]   7.3656453  17.9092897 270.6930871  11.5468718  14.3665453
0.2648049
## [13]   0.5860059   8.0483466   4.7964199   0.4752012   4.8757523
7.1460033
## [19]  59.9475608   0.6857300  25.7542601   3.2478494  17.4608856
0.9985205
## [25]  16.3546200  28.7284388   1.4709757   3.0699848   9.6750631
```

Indirection

We could use it *within* a mutate if we want a new column in our data frame:

```
murders ← murders %>%
  mutate(rate_sq = square(rate))
select(murders, starts_with("rate")) %>% head()
```

```
##       rate   rate_sq
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
## 6 1.292453  1.670435
```

But doing this for a lot of variables would require a lot of typing (and wouldn't vectorize over multiple variables well)

Indirection

What we might want to do is modify our function to use **variable names and the dataframe** as the arguments to directly add a new variable:

```
square_df ← function(var, # variable to square
                     df){ # data frame to square variables in
  df ← mutate(df,
              newvar = var * var)
  return(df)
}
```

Indirection

However, if we try and use this function on the `rate` variable in `murders` with a string, we get an error:

```
square_df(
  var = "rate",
  df = murders)
```

```
## Error in mutate():
## ℹ In argument: newvar = var * var.
## Caused by error in var * var:
## ! non-numeric argument to binary operator
```

Indirection

We get a similar error if we give the variable argument as a **data-variable**

- **data-variable**: a "statistical" variable that lives **in a data frame**
  - i.e. an object in the data frame (how we've been using variables in tidy verbs so far)

```
square_df(
  var = rate,
  df = murders)
```

```
## Error in mutate():
## ℹ In argument: newvar = var * var.
## Caused by error:
## ! object 'rate' not found
```

Indirection

This is an issue of **indirection**, which occurs in cases like this

- Want to interpret the argument as an **environment-variable** rather than as a **data-variable**.
- **env-variable**: "programming" variable/object that lives in your environment (i.e. data frame created with `<-`)

Fortunately, there are a couple programmatic ways around this.

Indirection

Solution A: provide the argument as a **data-variable**, then

1. **embrace** the argument with double braces `{{ arg }}` within function operations

```
square_embr ← function(var, # data-var rather than a string
                       df){
  df ← mutate(df,
              newvar = {{ var }} * {{ var }} )
  return(df)
}
square_embr(rate, murders) %>% select(rate, newvar) %>% head()
```

```
##       rate    newvar
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
## 6 1.292453  1.670435
```

Indirection

Solution B: provide the argument as a **data-variable**, and

1. **defuse** the string with `enquo()`
2. **inject** the defused string back in operations with `!! defused_string`

Let's work through the steps involved.

Defusing and Injecting

Defusing an argument with **delays its evaluation**.

- Use `enquo()` for defusing a single argmuent, `enquos()` for multiple simultaneously

For example, returning a defused variable highlights the different handling:

```
def_fun ← function(df, var){
  # defuse
  var ← enquo(var)
  return(var)
}
def_fun(murders, rate)
```

```
## <quosure>
## expr: ^rate
## env:  global
```

Defusing and Injecting

When we're ready to do something with the data-variable, we **inject** it back in

- Add `!!` in front of the data-variable in the operation

```
def_inj_fun ← function(df, var){
  # defuse
  var ← enquo(var)
  res ← pull(df, !!var)
  return(res)
}
def_inj_fun(murders, rate) %>% head()
```

```
## [1] 2.824424 2.675186 3.629527 3.189390 3.374138 1.292453
```

Defusing and Injecting

Using the two in our square function involves

1. Defuse the data-variable argument `var`
2. Inject it in the squaring operation with `!!`

```
square_def ← function(var, # data-var rather than
a string
                      df){
  var ← enquo(var) # defuse the string

  df ← mutate(df,
              newvar = !!var * !!var # square the
defused string
              )
  return(df)
}
square_def(rate, murders) %>% select(rate, newvar)
%>% head()
```

```
##       rate    newvar
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
## 6 1.292453  1.670435
```

Defuse + Inject

Solution C: defuse the argument with `ensym()`

- Allows for supplying the argument as either a **character string** or a **data variable**

```
square_ensym ← function(var, df){

  df ← mutate(df,
              newvar = !!ensym(var) * !!ensym(var) # square the defused
string
              )
  return(df)
}
```

Defuse + Inject

Solution C: defuse the argument with `ensym()`

- Allows for supplying the argument as either a **character string** or a **data variable**

```
square_ensym("rate", murders) %>% select(rate, newvar) %>% head(3)
```

```
##       rate   newvar
## 1 2.824424  7.97737
## 2 2.675186  7.15662
## 3 3.629527 13.17347
```

```
square_ensym(rate, murders) %>% select(rate, newvar) %>% head(3)
```

```
##       rate   newvar
## 1 2.824424  7.97737
## 2 2.675186  7.15662
## 3 3.629527 13.17347
```

ensym vs. enquo vs. {{ }}

`ensym()` and `enquo()` are two of the **tidy defusal operators**. But what's the difference?

- `ensym()` en"symbolizes" the argument and turns it into a **simple symbol**.
- `enquo()` converts the argument to a **quosure**

When combined with `!!` they allow for pretty advanced defusal operations.

Punchline: in most cases, **embracing** with `{{ }}` **works in most cases** and is **simpler**

Name Injection

We can combine defusing or embracing with **name injection** to customize our variable names.

- i.e. call the new squared rate variable `rate_sq` rather than `newvar`

Often we want to programmatically create new variable names based either on

1. A **supplied character string** as a function argument, or
2. **Modify the data-variable's name** directly in the function

Name Injection

Approach 1: use **glue syntax** and **supply the new name as a third argument**:

- argument `newname` the new variable's name (as a character string)

```
square_inj_1 ← function(var, df,
                        newname){ # new variable name to use


  return(df)
}
```

Name Injection

Approach 1: use **glue syntax** and **supply the new name as a third argument**:

- argument `newname` the new variable's name (as a character string)
- Glue syntax with `"{newname}"`

```
square_inj_1 ← function(var, df,
                        newname){ # new variable name to use

  df ← mutate(df,
              "{newname}"
  return(df)
}
```

Name Injection

Approach 1: use **glue syntax** and **supply the new name as a third argument**:

- argument `newname` the new variable's name (as a character string)
- Glue syntax with `"{newname}"`
- Programmatic assignment operator `:=` instead of `=`

```
square_inj_1 ← function(var, df,
                        newname){ # new variable name to use

  df ← mutate(df,
              "{newname}" :=
  return(df)
}
```

Name Injection

Approach 1: use **glue syntax** and **supply the new name as a third argument**:

- argument `newname` the new variable's name (as a character string)
- Glue syntax with `"{newname}"`
- Programmatic assignment operator `:=` instead of `=`
- Embracing `{{ }}` evaluates the argument contents in the context of the data frame

```
square_inj_1 ← function(df, var,
                        newname){ # new variable name to use

  df ← mutate(df,
              "{newname}" := {{ var }} * {{ var }} )
  return(df)
}

murders_exp ← square_inj_1(murders, total, "square_of_total") %>%
  square_inj_1(rate, "rate_squared") %>%
```

Name Injection

Approach 1: use **glue syntax** and **supply the new name as a third argument**:

- argument `newname` the new variable's name (as a character string)
- Glue syntax with `"{newname}"`
- Programmatic assignment operator `:=` instead of `=`
- Embracing `{{ }}` evaluates the argument contents in the context of the data frame

```
square_inj_1(murders, rate, "rate_sq") %>% select(rate, rate_sq) %>%
head()
```

```
##       rate   rate_sq
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
## 6 1.292453  1.670435
```

Name Injection

Approach 1 works with `ensym()` too

```
square_inj_1b ← function(var, df,
                         newname){ # new variable name to use

  df ← mutate(df,
              "{newname}" := !!ensym(var) * !!ensym(var) )
  return(df)
}
square_inj_1b("rate", murders, "rate_squared") %>% select(rate,
rate_squared) %>% head()
```

```
##       rate rate_squared
## 1 2.824424     7.977370
## 2 2.675186     7.156620
## 3 3.629527    13.173468
## 4 3.189390    10.172209
## 5 3.374138    11.384809
## 6 1.292453     1.670435
```

Name Injection

Approach 2A: use **glue syntax** and **create the name from the data-variable**:

- `expr()` "defuses" the supplied expression
  - Converts the data-variable (i.e. `rate`) to a name
- Glue syntax with `"{newname}"`
- Programmatic assignment operator `:=` instead of `=`

```
square_inj_2a ← function(var, df){
    new_var ← expr(rate) %>% paste0("_sq") # create new variable name internally

  df ← mutate(df,
              "{new_var}" := {{ var }} * {{ var }} ) # glue syntax to assign new name
  return(df)
}
square_inj_2a(rate, murders) %>% select(rate, rate_sq) %>% head()
```

```
##       rate   rate_sq
```

Name Injection

Approach 2B: use **glue syntax** and **embracing**:

- Glue syntax with `"{{newname}}_sq"` (no intermediate name object)
- Programmatic assignment operator `:=` instead of `=`

```
square_inj_2b ← function(var, df){

  df ← mutate(df,
              "{{ var }}_sq" := {{ var }} * {{ var }} ) # Glue syntax
without intermediate object
  return(df)
}
square_inj_2b(rate, murders) %>% select(rate, rate_sq) %>% head()
```

```
##       rate   rate_sq
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
```

Name Injection

Approach 2C: you guessed it, `ensym()` still works, allowing the `var` argument to be *either* a data-variable or string.

```
square_inj_2c ← function(var, df){

  df ← mutate(df,
              "{{ var }}_sq" := !!ensym(var) * !!ensym(var) ) # Glue
syntax without intermediate object
  return(df)
}
square_inj_2c("rate", murders) %>% select(rate, rate_sq) %>% head()
```

```
##       rate   rate_sq
## 1 2.824424  7.977370
## 2 2.675186  7.156620
## 3 3.629527 13.173468
## 4 3.189390 10.172209
## 5 3.374138 11.384809
## 6 1.292453  1.670435
```

Vectorization

Vectorization

Where the real benefits of custom functions, indirection, and name injection come in are with **vectorization** and **functionals**.

These approaches give a new way to repeatedly iterate a function over a vector of argument values.

Two main approaches:

1. **apply** family
   - `apply()`, `lapply()`, `sapply()`, `mapply()`
2. Tidy**map** **list** functions in **purrr**
   - `map()` and `map2()` with `list_c()`, `list_rbind()`, `list_cbind()`
     - Recently superseded the `map_dfr()`, `map_dfc()` functions

apply Family

The base R **apply** family gives methods for iterating a function over a vector of arguments depending on the format and type of output we want

Function	Description	Output Type
`lapply(X, FUN)`	apply `FUN` to every element of `X`	list
`sapply(X, FUN)`	apply `FUN` to every element of `X`	vector, matrix, or array
`vapply(X, FUN)`	`sapply` with specified output types	vector or array
`mapply(FUN, ARG1, ARG2, ... )`	multivariate version of `sapply`	list
`apply(X, MARGIN, FUN)`	apply `FUN` to every element of `X` over dimension `MARGIN`	vector, matrix, array, or list

Apply

Suppose you wanted to standardize all the numeric variables in the `murder` data.

You might write a function like this:

```
calculate_z = function(x) {
  z = (x - mean(x)) / sd(x)
  return(z)
}
```

apply Functions

However, applying it over all the numeric variables at once leads to this:

```
numeric_cols = c("total", "population", "rate")
murder_numbers = murders[numeric_cols]

calculate_z(murder_numbers)
```

```
## Error in is.data.frame():
## ! 'list' object cannot be coerced to type 'double'
```

This is an example of a function that **isn't vectorized.**

apply Functions

While we could put our function into a for loop, a more efficient/legible approach would use `sapply`[3]:

```
sapply(X, FUN)
```

```
sapply(murder_numbers, calculate_z) %>% head()
```

```
##            total  population        rate
## [1,] -0.2090939 -0.18890769  0.01844305
## [2,] -0.7003568 -0.78207215 -0.04231860
## [3,]  0.2017034  0.04609577  0.34623812
## [4,] -0.3869650 -0.46057478  0.16703781
## [5,]  4.5426034  4.54448196  0.24225740
## [6,] -0.5055457 -0.15254681 -0.60529343
```

[3] `sapply` is an example of a **functional:** a function that takes another function as an argument.

map and list_ Functions

The tidy alternative to the apply functions are the `map_` family in **purrr**

- Work a lot like the `apply_` functions, but with tidyverse syntax
- Combine with `list_` functions to convert to a vector or dataframe

Function	Description	Output Type
`map(X, FUN)`	apply `FUN` to every element of `X`	list
`map2(X1, X2, FUN)`	apply `FUN` to every element of `X1` and `X2`	list
`list_c()`	combine list elements into a vector	vector
`list_rbind()`	combines elements into a data frame row-wise	data frame
`list_cbind()`	combines elements into a data frame column-wise	data frame

map()

Just like with `sapply()` we can iterate our `calculate_z()` over all numeric variables:

`map(X, FUN)`

```
z_map ← map(murder_numbers, calculate_z)
class(z_map)
```

```
## [1] "list"
```

```
z_map
```

```
## $total
##  [1] -0.20909395 -0.70035678  0.20170342 -0.38696497  4.54260341
-0.50554566
##  [7] -0.37002488 -0.61989131 -0.36155483  2.05240907  0.81154693
-0.75117707
## [13] -0.73000195  0.76072663 -0.17944878 -0.69188673 -0.51401570
-0.28955941
```

list_

The `list_` functions provide a convenient way to convert `map()` output directly to a dataframe:

- Loop our `square_inj_2c()` function over all three numeric variables
- Combine each of the dataframes

```
map_sq ← map(
  c("total", "population", "rate"), # first argument: variable names
  square_inj_2c, # function to iterate over
  df = murders # additional static arguments
) %>%
  list_cbind(name_repair = "unique") # account for duplicated names
class(map_sq)
```

```
## [1] "data.frame"
```

```
colnames(map_sq)
```

```
##  [1] "state ... 1"          "abb ... 2"            "region ... 3"
```

Anonymous Functions

We can also use an anonymous (lambda) function in a `map()` too!

```
map(
  seq(1:20),
  .f = \(x) sqrt(x) + 3
) %>%
  unlist()
```

```
##  [1] 4.000000 4.414214 4.732051 5.000000 5.236068 5.449490 5.645751
5.828427
##  [9] 6.000000 6.162278 6.316625 6.464102 6.605551 6.741657 6.872983
7.000000
## [17] 7.123106 7.242641 7.358899 7.472136
```

Progress

Add the `.progress = TRUE` argument to any `map()` function to get a visible progress bar

```
map_many ← map(
  seq(1:2000000),
  .f = \(x) sqrt(x) + 3,
  .progress = T
) %>%
  unlist()
```

Alternative Arguments

Pass additional arguments to your function by name as new arguments of `map()`:

```
map(seq(1:10),
  log
) %>%
  unlist()
```

```
##  [1] 0.0000000 0.6931472
1.0986123 1.3862944 1.6094379
1.7917595 1.9459101
##  [8] 2.0794415 2.1972246
2.3025851
```

```
map(seq(1:10),
  log,
  base = 10
) %>%
  unlist()
```

```
##  [1] 0.0000000 0.3010300
0.4771213 0.6020600 0.6989700
0.7781513 0.8450980
##  [8] 0.9030900 0.9542425
1.0000000
```

Parallelization

Parallelization

One distinct advantage of R over Stata is the ability to **run code in parallel**

- i.e. split a repeated task across multiple CPU cores simultaneously
- Useful in any situation where we would use `map()` - i.e. bootstrapping, extracting parcel-level raster information

Stata

- SE: runs in "serial" on one core
- MP Student: 4 core ($375/yr)
- MP 8 Core: $655/yr

R and **furrr**

- `future_map` functions work exactly like **purrr's** `map()`
- Run across as many cores as your system has
- See progress with **progressr**
- Annual cost: $0

The Power of Parallel

To see the benefit of running code in parallel, let's write a **purposefully slow function:**

```
slow_square ← function(x = 1){
  Sys.sleep(1/2) # wait half a second
  return(x^2)
}
```

The Power of Parallel

How long does it take to run this function?[4]

- Use `tic()` and `toc()` from **tictoc** to calculate elapsed time

```
tic()
square_serial ← map(1:24, slow_square)
toc()
```

```
## 12.24 sec elapsed
```

The function runs in **serial**. so it takes approximately $1/2 * 24 = 12$ seconds

- Using one core, runs for $x = 1$, then when done moves on to $x = 2, \ldots, 24$

[4] `sapply()` and `map()` take nearly the exact same time. There are also several **type-specific versions** of `map` in case you want output to be a logical, integer, double, or character, etc.

Parallelization

We can **speed this up**. Modern CPUs are made up of multiple **cores** (processing units) that can all be given tasks simultaneously, allowing us to run code in **parallel**.

First, use `future::availableCores()` to determine how many cores you have:

```
availableCores()
```

```
## system
##     24
```

Your number of cores will likely differ

- Most laptops have at least 4-8 cores these days.
- Even recent Chromebooks have 6!

furrr

furrr functions make it easy to **parallelize** in just a few steps.

1. Set a "plan" for how the code will be run in parallel
    - Number of cores to use, how to execute tasks
2. Use `future_` version of your preferred `map_` function
3. Close parallel plan

First, we will **set the plan** and tell R how to execute the parallel session:

```
# Calculate a "safe" number of cores (allow for background processes)
n_cores = availableCores() - 2

# Set the "plan"
plan(strategy = "multisession", # run in parallel in separate background
R sessions
     workers = n_cores # use the desired number of cores
     )
```

furrr

furrr functions make it easy to **parallelize** in just a few steps.

1. Set a "plan" for how the code will be run in parallel
    - Number of cores to use, how to execute tasks
2. Use `future_` version of your preferred `map_` function
3. Close parallel plan

Next, let's repeat the previous analysis with `future_map()`.

```
tic()
square_parallel ← future_map(1:24, slow_square)
toc()
```

```
## 11.58 sec elapsed
```

furrr

furrr functions make it easy to **parallelize** in just a few steps.

1. Set a "plan" for how the code will be run in parallel
    - Number of cores to use, how to execute tasks
2. Use `future_` version of your preferred `map_` function
3. Close parallel plan

Now that we're done with our parallel session, reset things back to serial:

```
plan("sequential")
```

Benefits of Parallelization

Here we reduced execution time by ~ 1/3 due to some overhead of creating/assigning objects to the cores. However, the benefits of parallel increase substantially with

- Larger objects
- Greater number of repetitions (must be independent tasks)
- More cores

For example, if we run our slow function over the integers 1 to 1,000:

Approach	Time	Time Savings
Serial	1,017.3 Seconds	0%
Parallel, 5 cores	204.33 Seconds	80%
Parallel, 10 cores	103.87 Seconds	90%
Parallel, 20 cores	51.49 Seconds	95%

Built-In Progress

Like with `map()` functions, `future_map()` you can also add a progress bar with `.progress = T`

```
# Run over n_cores
plan(strategy = "multisession", # run in parallel in separate background
R sessions
     workers = n_cores # use the desired number of cores
     )

# Running over the same long vector in parallel:
tic()
square_parallel_long ← future_map(1:1000, slow_square, .progress = T)
toc()
```

```
## 34.02 sec elapsed
```

```
plan("sequential")
```

Progress with *progressr*

To **customize the progress-tracking process**, we can use the **progressr** package.[5]

This involves multiple steps, but gives us tons of customization options.

1. Add a progress indicator argument to your function
2. Build a wrapper function that creates a progress bar and vectorizes your function
3. Run your wrapper function within `with_progress()`

[5] Checking the documentation of `future_map()` notes that the `.progress` argument **will** be deprecated in the future, and recommends **progressr** methods.

Progress with *progressr*

First, let's **add a progress indicator argument** to our function and **increment the progress bar**

```
slow_square_prog ← function(x = 1, p){
  Sys.sleep(1/2) # wait half a second
  p() # increment the progress bar
  return(x^2)
}
```

Progress with *progressr*

Next, we need to actually create the progress bar with `progressor()` and vectorize our function in a **wrapper function**

```
wrap_slow_square ← function(x){
  # create the progress bar
  p ← progressor(along=x)
  # vectorize + pass progress bar as function argument
  res ← map(x, slow_square_prog, p = p)
  return(res)
}
```

Progress with *progressr*

Finally, run the wrapper function within `with_progress()`

```
with_progress(
  slow_square_vec ← wrap_slow_square(1:10) %>%
    unlist()
)
```

Tweaking Progress Bar

There are a lot of **different progress bar options**, including

- Continuous color bar

```
pacman::p_load(cli)
handlers("cli")

with_progress(wrap_slow_square(1:10))
```

Tweaking Progress Bar

There are a lot of **different progress bar options** we can tweak with `handlers()`, including

- Change the shape used in the ASCII progress bar

```
handlers(handler_txtprogressbar(char =
cli::col_red(cli::symbol$smiley)))

with_progress(wrap_slow_square(1:10))
```

Tweaking Progress Bar

There are a lot of **different progress bar options**, including

- Audible beeps at start, intervals, and finish

```
pacman::p_load(beepr)
handlers("cli", "beepr")

with_progress(wrap_slow_square(1:20))
```

Completion Notifications

There are a lot of **different progress bar options**, including

- Send notifications to your phone or desktop through **ntfy**

  1. Create personal, obscure ntfy topic and add it to your .Renviron

     - Topics aren't password-protected, so make secure
     - Here as an example I'm using `afre_ntfy`

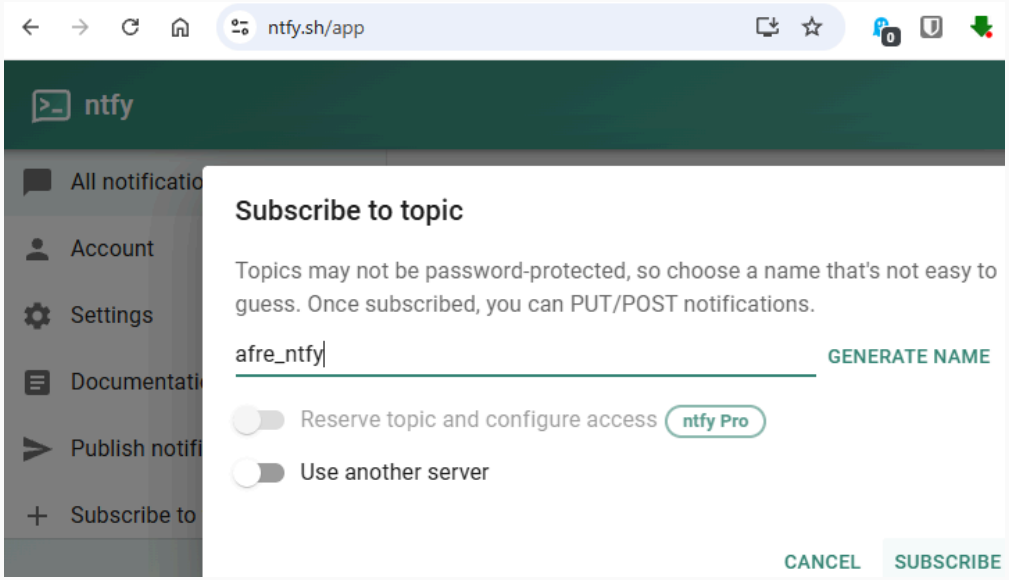
```
usethis::edit_r_environ()
# add a line with a custom topic
# NTFY_TOPIC='yourSecretTopic'
# read it in
readRenviron("~/Renviron")
```

Completion Notifications

There are a lot of **different progress bar options**, including

- Send notifications to your phone or desktop through **ntfy**

  2. Subscribe to the topic on the **phone or web app**



Completion Notifications

There are a lot of **different progress bar options**, including

- Send notifications to your phone or desktop through **ntfy**

  3. Send a message with `ntfy_send()`

```
pacman::p_load(ntfy)
ntfy_send("Hey this is nifty!")
```

```
handlers("cli", "ntfy")

with_progress(wrap_slow_square(1:10))
```

Tweaking Progress Bar

We can customize the sounds more fully with `handler_beepr()`:

```
sound_path ← paste0(getwd(), "/data/finish.wav")
handlers(list(
  "cli",
         handler_beepr(
           initiate = NA_integer_, # disable start sound
           update = NA_integer_, # disable progress sound
           finish = sound_path# set custom finish sound
            )
          )
        )
with_progress(wrap_slow_square(1:10))
```

Table of Contents

1. Prologue
2. If/Else Statements
3. For Loops
4. Functions
5. Indirection and Name Injection
6. Vectorization
7. Parallelization